

CSCE 620 COMPUTATIONAL GEOMETRY · MODULE 03 · LECTURE NOTES

Convex Hulls

The first real algorithm, and the first real degeneracy.

MODULE OUTCOMES

- Implement Graham scan and Andrew's monotone chain.
- Explain output-sensitive algorithms and the gift-wrapping bound.
- State and justify the lower bound for convex hull.
- Handle collinear and duplicate points deliberately.
- Extend to three dimensions and state what changes.

1 The planar algorithms

```
std::sort(P.begin(), P.end()); // lexicographic
P.erase(std::unique(P.begin(), P.end()), P.end()); // duplicates FIRST

std::vector<Pt> h(2*P.size()); int k = 0;
for (int i = 0; i < P.size(); i++) { // lower hull
    while (k >= 2 && orient(h[k-2], h[k-1], P[i]) <= 0) k--;
    h[k++] = P[i];
}
for (int i = P.size()-2, t = k+1; i >= 0; i--) { // upper hull
    while (k >= t && orient(h[k-2], h[k-1], P[i]) <= 0) k--;
    h[k++] = P[i];
}
h.resize(k-1);
```

$O(n \log n)$, dominated entirely by the sort — the two sweeps are linear, because each point is pushed once and popped at most once. Twenty lines, one predicate, and **one genuinely consequential comparison operator**.

THE COLLINEAR DECISION IS A SPECIFICATION, NOT A DETAIL

With ≤ 0 , collinear points lying on a hull edge are popped and discarded. The result is the minimal set of vertices — the extreme points, and nothing else.

With < 0 , they are kept. The result includes every input point lying anywhere on the hull boundary, including the interiors of edges.

Both are correct convex hulls. They are different answers because they are answers to different questions, and neither is a bug.

And the caller cares which one it gets. A renderer drawing the hull outline wants the minimal set. A mesh-boundary extractor that must map every input vertex to a boundary position needs all of them. A collision proxy generator wants the minimal set for fewer SAT axes. **This is the most common silent mismatch between a hull routine and its caller** — decide it deliberately, document it at the interface, and test both behaviours.

Algorithm	Cost	Character
Andrew's monotone chain	$O(n \log n)$	The simplest correct implementation, and the one to use. Lexicographic sort, two linear sweeps, no pivot, no angles.
Graham scan	$O(n \log n)$	Sorts by angle about an extreme point. Historically first, and it carries degenerate cases the monotone chain does not — see below.
Gift wrapping (Jarvis march)	$O(nh)$	Output-sensitive. Finds one hull vertex per pass over all n points. Excellent when h is small, quadratic when it is not.
QuickHull	$O(n \log n)$ expected	$O(n^2)$ worst case , but very fast in practice and it generalises cleanly to 3D. The usual choice in graphics libraries.
Divide and conquer	$O(n \log n)$	Split, recurse, merge by finding the two bridging tangents. The route that generalises to three dimensions.
Chan's algorithm	$O(n \log h)$	Optimal. Runs gift wrapping over the hulls of small groups, doubling a guess at h until it succeeds. Beautiful, and rarely worth the implementation effort.

h is the number of hull vertices, and its growth matters for the output-sensitive bounds: for points uniform in a disc, h grows like $n^{1/3}$; for points uniform in a square, like $\log n$; for points on a circle, $h = n$. **So the output-sensitive algorithms win on realistic input and lose on the adversarial case**, which is the usual shape of that trade.

WHY GRAHAM SCAN IS THE WORSE CHOICE DESPITE BEING THE FAMOUS ONE

It sorts by angle about an extreme point, which requires choosing a pivot — and ties in angle must then be broken by distance from that pivot, in a direction that differs between the first and last edge.

Points coincident with the pivot, and multiple points at identical angles, are both degenerate cases that the monotone chain does not have at all. They are not hard to handle; they are simply extra code and extra opportunities to be wrong.

And angle comparison invites `atan2`, which is slow, transcendental, and immediately destroys any exactness you established in Module 02. The correct implementation compares angles using `orient` instead — but the obvious implementation does not, and the obvious one is what gets written.

The monotone chain sorts lexicographically: exact, total, no pivot, no ties requiring geometric reasoning. Same asymptotic bound, strictly fewer ways to be wrong — which is the right basis for choosing between two algorithms of equal complexity.

2 Lower bound and output sensitivity

$$x_i \rightarrow p_i = (x_i, x_i^2)$$

Every such point lies on the parabola $y = x^2$, which is strictly convex, **so every point is a hull vertex**. Reading the hull in order around its boundary therefore yields the x_i in sorted order. **A convex hull algorithm sorts**, so the $\Omega(n \log n)$ comparison-model lower bound for sorting transfers directly — and the monotone chain matches it, which closes the planar problem. **The parabola reduction is worth remembering**; it recurs in lower-bound arguments throughout computational geometry, and it is the same reduction technique CSCE 629 developed.

OUTPUT-SENSITIVE ALGORITHMS CHARGE BY THE ANSWER

Gift wrapping is $O(nh)$: it finds hull vertices one at a time, scanning all n points to determine the next one, and so pays n per vertex produced.

When h is small this comfortably beats $O(n \log n)$. Ten thousand points whose hull happens to be a triangle costs about 30,000 operations, against roughly 130,000 for the sort — and no sorting at all, which matters more than the operation count suggests because the memory traffic is sequential.

When $h = n$ it degenerates to $O(n^2)$, which is far worse. Points on a circle — a perfectly ordinary input, produced by any radial sampling — are the bad case.

Chan's algorithm achieves $O(n \log h)$, which is optimal and dominates both. It partitions the points into groups of size m , hulls each group by divide and conquer, then gift-wraps over the group hulls; m is a guess at h , doubled (squared, in fact) until the wrap completes. **Genuinely elegant, and rarely worth implementing** — the constant factors and the code complexity usually lose to the monotone chain in practice.

3 Three dimensions

Aspect	Two dimensions	Three dimensions
Output size	h vertices, in a cycle.	$O(n)$ vertices, edges, and faces — Euler's formula $V - E + F = 2$ keeps it linear, which is not obvious in advance.
Predicate	orient2d, a 2×2 determinant.	orient3d, a 3×3 determinant — the signed volume of a tetrahedron. Same filtering treatment as Module 02.
Structure	A cyclic sequence of vertices.	A polyhedron , requiring a half-edge or winged-edge representation to navigate adjacency — the structure CSCE 645 built.
Hard degeneracy	Collinear points.	Coplanar points , which are far more common in real meshes and considerably worse: four coplanar points admit no unique face, so the algorithm must choose, and inconsistent choices produce a non-closed surface.

Aspect	Two dimensions	Three dimensions
Usual method	Monotone chain.	Incremental (add points one at a time, delete visible faces, cone to the horizon) or QuickHull3D . Divide and conquer also works and is harder to implement.
Four or more dimensions	—	Output size can be $O(n^{d/2})$ by the upper bound theorem — exponential in the dimension. This is why nobody computes hulls in high dimensions, and why the curse of dimensionality shows up again in Module 10.

THE ENGINE CONNECTION

Collision detection operates on convex shapes. GJK (Module 12) requires convexity and essentially nothing else — no explicit face list, just a support function.

So concave meshes are decomposed into convex pieces offline, and each piece's convex hull becomes its collision proxy. Approximate convex decomposition (V-HACD and relatives) is the standard asset-pipeline step, and it is built on repeated hull computation.

Hulls also give bounding volumes, k-DOPs, and the candidate separating axes for the separating axis theorem — the face normals and edge cross-products of the hull are exactly the axes SAT must test.

And CSCE 649 assumed all of this already existed. That course took collision geometry as given so it could get to the dynamics; **this is where the geometry it relied on actually comes from**, including the robustness properties that determine whether the simulation is stable.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Mount — CMSC 754, convex hull lectures (free PDF)	Monotone chain, gift wrapping, the lower bound, and Chan's algorithm, all done properly.
Barber, Dobkin & Huhdanpaa — The Quickhull Algorithm for Convex Hulls (free)	The paper and the qhull implementation, which is what most software actually calls in 3D and higher.
Chan — Optimal output-sensitive convex hull algorithms (free)	The $O(n \log h)$ result of §2, from its author. Short.
Ericson — Real-Time Collision Detection, chapters on convexity and bounding volumes	The §3 engine connection, in shipping terms.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement Andrew's monotone chain using your Module 02 predicate.
- 2 **Implement both collinear policies** behind a flag, and produce an input where they differ. Draw both.
- 3 Test on: all points identical, all collinear, a single point, two points, and the empty set. Fix what breaks.
- 4 Implement gift wrapping and measure the crossover against monotone chain as h varies.
- 5 **Measure h empirically** for points uniform in a disc and in a square, over several n . Compare against $n^{1/3}$ and $\log n$.
- 6 Implement Graham scan and count how many degenerate cases it needs that the monotone chain does not.
- 7 **Verify the parabola reduction** by sorting a list through your hull routine.
- 8 Implement an incremental 3D convex hull with `orient3d`.
- 9 **Feed it coplanar input** — a cube's vertices, a flat grid — and report what happens.
- 10 Compare your 3D hull against `qhull` on correctness and speed.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Write the monotone chain and state its cost and what dominates it.
- 2 What does the collinear comparison decide, and who cares?
- 3 Give three reasons to prefer monotone chain over Graham scan.
- 4 State the lower bound reduction for convex hull.
- 5 What is an output-sensitive bound? Give the gift-wrapping and Chan bounds.
- 6 How does h grow for points in a disc, in a square, and on a circle?
- 7 Name five things that change in three dimensions.
- 8 Why is coplanarity worse than collinearity?
- 9 How does a physics engine use convex hulls?